

视频生成模型 VAE

1. 压缩参数与 DiT token 数

倍率按时间 $\times$ 高度 $\times$ 宽度排列， C 为 VAE 通道数。末列为 DiT patchify 后、投影前的形状，patch 内元素并入通道维。

$$R_{\text{nom}} = \frac{3r_t r_h r_w}{C}, \quad R_{\text{actual}} = \frac{3THW}{CT_z H_z W_z}, \quad N = \frac{T_z H_z W_z}{p_t p_h p_w}.$$

DiT patch 为 $p_t \times p_h \times p_w$ ，只重排元素；压缩比按 VAE 输出计算，已计入 VAE 内部 patchify。

模型	VAE	时间 $\times$ 高 $\times$ 宽压缩	Latent 通道 C	元素压缩比 R_{nom}	DiT patchify	DiT 输入形状
MiniMax H3	H3-VisualVAE 因果 3D CNN encoder + 非因果 ViT decoder；一次投影恢复时空像素块。	$4 \times 16 \times 16$	24	128:1	$1 \times 2 \times 2$	$[B, 96, T_{H3}, \frac{H}{32}, \frac{W}{32}]$
LTX-2.5	LTX-2.5 Video VAE 因果 3D CNN encoder + 非因果 3D CNN / NA Transformer 扩散 decoder (CNN 按默认配置)；空间 patchify $\times 4$ ；扩散版单步像素去噪。	$8 \times 32 \times 32$	128	192:1	$1 \times 1 \times 1$	$[B, 128, 1 + \frac{T-1}{8}, \frac{H}{32}, \frac{W}{32}]$
FLUX 3 Action	FLUX 3 Video VAE 因果 NA Transformer encoder + 非因果 NA Transformer decoder； $5 \times 5 \times 5$ 邻域 attention。	$4 \times 32 \times 32$	96	128:1	$1 \times 1 \times 1$	$[B, 96, 1 + \frac{T-1}{4}, \frac{H}{32}, \frac{W}{32}]$
Wan 2.1 / 2.2 A14B / Lingbot-Video	Wan2.1-VAE 因果 3D CNN encoder + 因果 3D CNN decoder；帧内空间 attention、逐层历史缓存。	$4 \times 8 \times 8$	16	48:1	$1 \times 2 \times 2$	$[B, 64, 1 + \frac{T-1}{4}, \frac{H}{16}, \frac{W}{16}]$
Wan 2.2 TI2V-5B	Wan2.2-VAE 因果 3D CNN encoder + 因果 3D CNN decoder；空间 patchify $\times 2$ 、残差采样 shortcut。	$4 \times 16 \times 16$	48	64:1	$1 \times 2 \times 2$	$[B, 192, 1 + \frac{T-1}{4}, \frac{H}{32}, \frac{W}{32}]$
HunyuanVideo-1.5	AutoencoderKLConv3D 因果 3D CNN encoder + 因果 3D CNN decoder；瓶颈因果 attention、残差式 3D pixel shuffle。	$4 \times 16 \times 16$	32	96:1	$1 \times 1 \times 1$	$[B, 32, 1 + \frac{T-1}{4}, \frac{H}{16}, \frac{W}{16}]$
MAGI-2 Preview	Wan2.2-VAE + TurboVAED 因果 3D CNN encoder + 非因果 3D CNN 蒸馏 decoder。	$4 \times 16 \times 16$	48	64:1	$1 \times 1 \times 1$	$[B, 48, 1 + \frac{T-1}{4}, \frac{H}{16}, \frac{W}{16}]$

H3 的时间压缩比较特殊，严格来说并不是 4:1，元素压缩比也要比 128:1 小一些。

2. Wan2.1-VAE

接着我们具体介绍一下应该是最经典的 VAE: Wan2.1-VAE。以 81 帧、480×832 的 RGB 视频为例：
整段视频: [3, 81, 480, 832] → **latent**: [16, 21, 60, 104] → 重建视频: [3, 81, 480, 832]。

$$T_z = 1 + \frac{81 - 1}{4} = 21, \quad H_z = \frac{480}{8} = 60, \quad W_z = \frac{832}{8} = 104.$$

2.1 Encoder: 跟踪后续的 4 帧视频块

整段视频按 1、4、4.....帧分成 21 个块: 1 个首帧块, 加上 20 个后续块, Wan2.1-VAE 会做 Feature Cache, 保留前面块的历史特征。下面我们用一个后续块来看看 Wan2.1-VAE 的具体 forward 过程:

顺序	模块与操作	输出形状
输入	当前视频块	[3, 4, 480, 832]
入口	因果卷积 3→96	[96, 4, 480, 832]
第一级	残差块 ×2, 输出 96 通道 → 空间下采样	[96, 4, 240, 416]
第二级	残差块 ×2, 输出 192 通道 → 时空下采样	[192, 2, 120, 208]
第三级	残差块 ×2, 输出 384 通道 → 时空下采样	[384, 1, 60, 104]
第四级	残差块 ×2, 输出 384 通道	[384, 1, 60, 104]
瓶颈	残差块 → 空间 Attention → 残差块	[384, 1, 60, 104]
出口	RMSNorm → SiLU → 因果卷积 384→32	[32, 1, 60, 104]

全部 21 个块编码完成后, 先沿时间维拼接, 经过一个 1×1×1 卷积, 再拆分为 μ 和 $\log_var$, 各为 [16, 21, 60, 104], 对 μ 按通道标准化后作为输入。

2.2 Decoder: 跟踪后续的 latent 时间步

先对完整 latent 撤销标准化, 经过 1×1×1 卷积, 16→16, 形状仍为 [16, 21, 60, 104]。随后逐个 latent 时间位置解码; 下表跟踪一个后续时间步:

顺序	模块与操作	输出形状
输入	撤销标准化后的 latent 块	[16, 1, 60, 104]
入口	因果卷积 16→384	[384, 1, 60, 104]
瓶颈	残差块 → 空间 Attention → 残差块	[384, 1, 60, 104]
第一级	残差块 ×3, 输出 384 通道 → 时空上采样, 384→192	[192, 2, 120, 208]
第二级	残差块 ×3, 先将 192 升至 384 → 时空上采样, 384→192	[192, 4, 240, 416]
第三级	残差块 ×3, 输出 192 通道 → 空间上采样, 192→96	[96, 4, 480, 832]
第四级	残差块 ×3, 输出 96 通道	[96, 4, 480, 832]
出口	RMSNorm → SiLU → 因果卷积 96→3	[3, 4, 480, 832]

其中, 首帧会跳过时间重采样, 只输出 1 帧。

2.3 上下采样

操作	具体实现	形状变化
空间下采样	逐帧右侧、下侧补 1 $\rightarrow 3 \times 3$ Conv2d, stride=2	H, W 各减半; C, T 不变
时间下采样	拼接 1 帧历史缓存 $\rightarrow 3 \times 1 \times 1$ 时间卷积, 时间步长 2	后续块 T 减半; C, H, W 不变
空间上采样	逐帧最近邻插值 $\times 2$ (nearest-exact) $\rightarrow 3 \times 3$ Conv2d, $C \rightarrow C/2$	H, W 各翻倍, C 减半; T 不变
时间上采样	$3 \times 1 \times 1$ 因果卷积, $C \rightarrow 2C \rightarrow$ 将两组通道重排到时间维	后续块 T 翻倍; 最终 C, H, W 不变

执行顺序：时空下采样是 先空间、后时间；时空上采样是 先时间、后空间。

残差块、缓存与 Attention

- 残差块：主分支执行两组 “RMSNorm $\rightarrow$ SiLU $\rightarrow 3 \times 3 \times 3$ 因果卷积”，再加 shortcut；通道不同时，shortcut 通过 $1 \times 1 \times 1$ 卷积对齐。残差块保持 T, H, W 。
- 因果卷积缓存：普通时间核为 3、步长为 1 的因果卷积，需要前 2 个时间位置的特征；序列开头不足处补零。缓存由各层分别维护，时间位置以该层分辨率为准。上表时间下采样使用的是单独的 1 帧缓存路径。
- 帧内空间 Attention：只在同一时间位置的二维特征图上计算，保持 $[C, T, H, W]$ ；跨时间的信息通过因果卷积传递。

以 Wan2.1-VAE 为基础，我们简单介绍一下其他后续模型。

Wan2.2-VAE 延续了 Wan2.1-VAE 的因果卷积和 $4n + 1$ 处理方式，时间压缩倍率仍为 4。主要变化是入口新增 2×2 patchify，将空间位置重排到通道维，再经过三次空间下采样，使空间压缩从 8×8 提升到 16×16 ，同时将 latent 通道数从 16 增加到 48。结构上，Wan2.2 加宽了编码器和解码器：编码器基础通道数从 96 增至 160，瓶颈通道数从 384 增至 640；解码器基础通道数从 96 增至 256，瓶颈通道数从 384 增至 1024。同时，在原有残差块之外，为整个采样级增加 shortcut：下采样通过重排与 Avgpool 对齐形状，上采样通过重排与 copy 对齐形状，再与主分支相加。

HunyuanVideo-1.5 保持 $4 \times$ 时间压缩，将空间高宽各自的压缩倍率从 $8 \times$ 提高到 $16 \times$ ，latent channels 从 16 增至 32。相比 Wan 的分离式时空采样，它采用 causal convolution 加 PixelShuffle/Unshuffle 式重排，并在采样模块和 latent 两端的投影中增加 residual shortcuts。中间 Attention 则从帧内的 spatial attention 扩展为 spatiotemporal causal attention，能够直接关注当前及历史帧，实现跨帧信息交互。

3. MiniMax-H3 VAE

接着来看 MiniMax-H3 VAE。MiniMax H3 的默认视频分块规则要求长度对齐到 $17n + 5$ （H3 在这方面的处理非常神奇，虽然我想过一些可能的原因，但是还没找到官方依据），如果原视频是 81 帧 ($4d + 1$)，本例先复制末帧补到 90 帧，重建后裁掉末尾 9 帧。

整段视频：[3, 90, 480, 832] → latent：[24, 27, 30, 52] → 重建视频：[3, 90, 480, 832]。

$$90 = 17 \times 5 + 5, \quad T_z = 5 \times 5 + 2 = 27, \quad H_z = \frac{480}{16} = 30, \quad W_z = \frac{832}{16} = 52.$$

3.1 Encoder：跟踪一个 17 帧视频块

编码时，90 帧还会在内部复制末帧，临时补到 102 帧，再分成 6 个 17 帧块独立编码。

顺序	模块与操作	输出形状
输入	当前视频块	[3, 17, 480, 832]
入口	$3 \times 3 \times 3$ 因果卷积， $3 \rightarrow 128$	[128, 17, 480, 832]
第一级	残差块 $\times 2$ ，输出 128 通道 → 空间下采样	[128, 17, 240, 416]
第二级	残差块 $\times 2$ ，输出 256 通道 → 时空下采样	[256, 9, 120, 208]
第三级	残差块 $\times 2$ ，输出 256 通道 → 时空下采样	[256, 5, 60, 104]
第四级	残差块 $\times 2$ ，输出 512 通道 → 空间下采样	[512, 5, 30, 52]
第五级	残差块 $\times 2$ ，输出 512 通道	[512, 5, 30, 52]
第六级	残差块 $\times 2$ ，输出 1024 通道	[1024, 5, 30, 52]
出口	GroupNorm → SiLU → 因果卷积， $1024 \rightarrow 48$	[48, 5, 30, 52]
分布投影	$1 \times 1 \times 1$ 卷积， $48 \rightarrow 48$	[48, 5, 30, 52]

两次时间下采样均向上取整，时间长度依次为 $17 \rightarrow 9 \rightarrow 5$ 。全部 6 个块编码完成后，沿时间维拼接得到 [48, 30, 30, 52]，再按 token_drop=3 从整段末尾丢弃 3 个时间位置，得到 [48, 27, 30, 52]（丢弃的部分对应复制的 12 个末帧）。

随后拆分为 μ 和 $\log_var$ ，各为 [24, 27, 30, 52]；从后验分布采样得到 latent，再按通道标准化。

3.2 Decoder：跟踪一个 latent 窗口

先对完整 latent 撤销标准化，再按 7 个时间位置一窗、每次前进 5 个位置解码，相邻窗口重叠 2 个位置。下面跟踪其中一个 [24, 7, 30, 52] 的窗口；其视频 token 数和隐藏维度分别为：

$$N = 7 \times 30 \times 52 = 10920, \quad D = 32 \times 64 = 2048.$$

顺序	模块与操作	输出形状
输入	撤销标准化后的 latent 窗口	[24, 7, 30, 52]
入口	$1 \times 1 \times 1$ 卷积， $24 \rightarrow 24$	[24, 7, 30, 52]
展平	每个时空位置转为一个 token	[10920, 24]
特征投影	Linear， $24 \rightarrow 2048$	[10920, 2048]
附加 token	追加 4 个 register token + 1 个初始为零的 token（牢 CV 操作）	[10925, 2048]
主体	Transformer block $\times 36$ ，3D RoPE	[10925, 2048]

顺序	模块与操作	输出形状
出口	LayerNorm $\rightarrow$ Linear, 2048$\rightarrow$3072	[10925, 3072]
去除附加 token	保留视频 token	[10920, 3072]
像素重排	每个 token 展开成 4$\times$16$\times$16 RGB 块	[3, 28, 480, 832]

出口将每个视频 token 投影为 $3072 = 3 \times 4 \times 16 \times 16$ 个值，再重排成一个 $4 \times 16 \times 16$ RGB 块。因此，一个窗口先得到 [3, 28, 480, 832]，随后还要做时间裁剪和窗口融合。

时间裁剪与整段拼接

每个窗口输出的 28 帧先分成前 20 帧和后 8 帧，两部分各裁掉开头 3 帧，留下 17 帧主体 + 5 帧重叠区域。

27 个 latent 时间位置共形成 5 个窗口。从 0 开始计数，窗口范围依次为 0-6、5-11、10-16、15-21、20-26。每窗保留 22 帧，相邻窗口融合重叠的 5 帧，最终得到：

$$5 \times 22 - 4 \times 5 = 90 \text{ 帧.}$$

融合后的形状为 [3, 90, 480, 832]。再裁掉本例额外补入的末尾 9 帧，即恢复为 [3, 81, 480, 832]。

3.3 下采样与像素展开

操作	具体实现	形状变化
空间下采样	右、下各反射补 1 $\rightarrow 3 \times 3 \times 3$ 因果卷积，步长 (1, 2, 2)	H, W 各减半； C, T 不变
时空下采样	右、下各反射补 1 $\rightarrow 3 \times 3 \times 3$ 因果卷积，步长 (2, 2, 2)	H, W 各减半； $T \rightarrow \lceil T/2 \rceil$ ； C 不变
像素展开	Linear $\rightarrow$ 时空重排	每个视频 token 输出一个 $4 \times 16 \times 16$ RGB 块

下采样顺序：空间 $\rightarrow$ 时空 $\rightarrow$ 时空 $\rightarrow$ 空间，分别位于前四级末尾。两种下采样中的时间卷积都在左侧补 2 个零位置。

残差块、归一化与 Attention

- 残差块：主分支执行两组 “GroupNorm $\rightarrow$ SiLU $\rightarrow 3 \times 3 \times 3$ 因果卷积”，再加 shortcut；通道不同时，shortcut 通过 $1 \times 1 \times 1$ 卷积对齐。残差块保持 T, H, W 。
- 因果编码：时间卷积只在左侧补零，GroupNorm 按时间位置独立计算。各个 17 帧块独立进入 Encoder。
- ViT 解码块：依次执行 RMSNorm $\rightarrow$ Attention，以及 RMSNorm $\rightarrow$ 门控 SiLU FFN；两个分支分别经过可学习缩放后加回残差，保持 [N, 2048]。
- Attention 范围：发布配置中的 ViT Decoder 为非因果结构，可以同时关注当前窗口内的各个时间位置。

4. LTX-2.5 Video VAE

接着来看 LTX-2.5 的视频 VAE。它采用因果 **3D CNN Encoder**，提供两种 Decoder：非因果 **3D CNN** 解码器和非因果 **NA Transformer** 扩散解码器。两者使用相同的 latent 空间，时间压缩倍率为 8，空间高宽各压缩 32 倍，latent 通道数为 128。

同样以 81 帧、480×832 的 RGB 视频为例，形状统一写作 $[C, T, H, W]$ ，省略 $B=1$ ：

整段视频： $[3, 81, 480, 832] \rightarrow \text{latent} : [128, 11, 15, 26] \rightarrow \text{重建视频} : [3, 81, 480, 832]$ 。

$$T_z = 1 + \frac{81 - 1}{8} = 11, \quad H_z = \frac{480}{32} = 15, \quad W_z = \frac{832}{32} = 26.$$

下面跟踪整段视频的 forward 过程。分块与重叠融合属于另外的执行策略，不将视频预先视为 11 个独立编码、解码的小块。

4.1 Encoder：跟踪整段 81 帧视频

入口先做 **4×4 空间 patchify**：将每帧相邻的 16 个像素位置重排到通道维，时间长度不变。随后经过 **空间 → 时间 → 时空 → 时空四级下采样**。

顺序	模块与操作	输出形状
输入	RGB 视频	$[3, 81, 480, 832]$
Patchify	空间 4×4 重排， 3→48	$[48, 81, 120, 208]$
入口	3×3×3 因果卷积，48→128	$[128, 81, 120, 208]$
第一级	残差块 ×4，128 通道 → 空间下采样， 128→256	$[256, 81, 60, 104]$
第二级	残差块 ×6，256 通道 → 时间下采样， 256→512	$[512, 41, 60, 104]$
第三级	残差块 ×4，512 通道 → 时空下采样， 512→1024	$[1024, 21, 30, 52]$
第四级	残差块 ×2，1024 通道 → 时空下采样， 1024→1024	$[1024, 11, 15, 26]$
瓶颈	残差块 ×2，1024 通道	$[1024, 11, 15, 26]$
出口	PixelNorm → SiLU → 因果卷积， 1024→129	$[129, 11, 15, 26]$

三次时间下采样均保留首帧对应的位置，时间长度依次为：

$$81 \rightarrow 41 \rightarrow 21 \rightarrow 11.$$

卷积头输出的前 128 个通道是 μ 。类似地，再使用逐通道均值和标准差归一化，得到 $[128, 11, 15, 26]$ 。只是这里相比其他的 VAE 没有额外的 $1 \times 1 \times 1$ 分布投影卷积。

4.2 CNN Decoder: 逐级恢复视频

先撤销 latent 的逐通道归一化，再经过卷积残差块和四级上采样，最后通过 unpatchify 恢复 RGB 像素。

顺序	模块与操作	输出形状
输入	撤销归一化后的 latent	[128, 11, 15, 26]
入口	$3 \times 3 \times 3$ 卷积, 128 → 1024	[1024, 11, 15, 26]
第一级	残差块 $\times 2$, 1024 通道 → 时空上采样, 1024 → 512	[512, 21, 30, 52]
第二级	残差块 $\times 2$, 512 通道 → 时空上采样, 512 → 512	[512, 41, 60, 104]
第三级	残差块 $\times 4$, 512 通道 → 时间上采样, 512 → 256	[256, 81, 60, 104]
第四级	残差块 $\times 6$, 256 通道 → 空间上采样, 256 → 128	[128, 81, 120, 208]
第五级	残差块 $\times 4$, 128 通道	[128, 81, 120, 208]
出口	PixelNorm → SiLU → $3 \times 3 \times 3$ 卷积, 128 → 48	[48, 81, 120, 208]
Unpatchify	将通道重排成 4×4 空间像素块, 48 → 3	[3, 81, 480, 832]

时间上采样先将长度翻倍，再删除最前面的一个时间位置，因此每次为 $T \rightarrow 2T - 1$ ：

$$11 \rightarrow 21 \rightarrow 41 \rightarrow 81.$$

该 Decoder 的卷积以非因果模式执行，能够利用前后时间位置的信息。

4.3 扩散 Decoder: 条件特征与像素去噪

它先将 latent 上采样为条件特征 **context**，再以 context 为条件，对像素噪声进行重建，1 次去噪、直接预测重建视频 x_0 。

生成条件特征

官方实现会将最后一个 latent 时间位置额外复制 2 次，用于处理邻域 Attention 的末端边界。我们计入这部分临时补帧，并在最后裁掉。

顺序	模块与操作	输出形状
输入	撤销归一化后的 latent	[128, 11, 15, 26]
边界处理	将最后一个 latent 时间位置额外复制 2 次	[128, 13, 15, 26]
入口	逐位置 Linear, 128 → 2048	[2048, 13, 15, 26]
第一级	NA Block $\times 4$ → 空间上采样, 2048 → 1024	[1024, 13, 30, 52]
第二级	NA Block $\times 6$ → 时间上采样, 1024 → 512	[512, 25, 30, 52]
第三级	NA Block $\times 4$ → 时空上采样, 512 → 512	[512, 49, 60, 104]
第四级	NA Block $\times 2$ → 时空上采样, 512 → 256	[256, 97, 120, 208]
裁剪	去掉临时补帧对应的末尾 16 个时间位置	[256, 81, 120, 208]

补入的 2 个 latent 时间位置经过 8 倍时间展开，对应 16 个时间位置，因此裁剪后为 $97 - 16 = 81$ 。得到的 context 已经具有目标视频的时间长度，空间分辨率仍是目标的 $1/4 \times 1/4$ 。

像素去噪与重建

初始化与目标视频同形状的随机像素噪声。下面跟踪像素特征分支，context 在各个扩散块中作为条件注入。

顺序	模块与操作	输出形状
输入	随机像素噪声 x_t	[3, 81, 480, 832]
Patchify	空间 4×4 重排, 3→48	[48, 81, 120, 208]
入口	逐位置 Linear, 48→256	[256, 81, 120, 208]
去噪主干	Diffusion NA Block ×8, 注入 context 和去噪时间步条件	[256, 81, 120, 208]
出口	RMSNorm → Linear, 256→48	[48, 81, 120, 208]
Unpatchify	将通道重排成 4×4 RGB 像素块, 48→3	[3, 81, 480, 832]

最后输出重建视频 x_0 。这里的 8 个 **Block** 是网络深度；“去噪时间步”表示噪声等级，与视频的 81 个时间位置是不同概念。

4.4 NA Attention: 局部时空注意力

NA (Neighborhood Attention) 就是邻域注意力。对于特征图上的每个位置 (t, h, w)，只取附近一个三维窗口中的 K、V，与当前位置的 Q 计算 Attention。窗口随查询位置滑动，相邻窗口重叠。

例如 3×7×7 窗口，在非边界处覆盖前一个、当前、后一个时间位置，每个时间位置取 7×7 个空间格点。因此，每个查询、每个注意力头关注 $3 \times 7 \times 7 = 147$ 个位置，包括自身。

阶段	特征通道	Attention 头数	每头维度	窗口: 时间 × 高 × 宽	每头关注的位置数
第一级	2048	32	64	3×7×7	147
第二级	1024	16	64	3×7×7	147
第三级	512	8	64	3×5×5	75
第四级	512	8	64	3×5×5	75
第五级: 像素去噪	256	4	64	11×11×11	1331

所有阶段的每头维度都是 **64**，头数为 $C/64$ 。窗口大小按所在层的特征格点计算；NA 本身保持 [C, T, H, W]，上采样由另外的模块完成。

普通 NA Block 依次执行：

```
RMSNorm -> 三维 NA Attention -> 加残差
RMSNorm -> SwiGLU MLP -> 加残差
```

Q、K 使用每头 RMSNorm 和三维 RoPE；MLP 隐藏维度为 **4C**。第五级的 Diffusion NA Block 额外注入 context，并通过去噪时间步调制特征。

这些 Attention 都是非因果的，允许关注后续时间位置。到达边界时，窗口向有效区域内平移以保持大小，例如时间窗口为 3 时，首位置关注 0、1、2。

4.5 上下采样与时间边界

操作	具体实现	形状变化
空间 Patchify	将每帧 4×4 像素块重排到通道维	H, W 各除以 4; C 乘以 16; T 不变
空间下采样	因果卷积 → 空间重排到通道; 加重排、分组平均得到的 shortcut	H, W 各减半; T 不变
时间下采样	复制首帧 1 次 → 因果卷积与时间重排; 加 shortcut	$T \rightarrow (T + 1)/2$; H, W 不变
时空下采样	复制首帧 1 次 → 因果卷积与 2×2×2 联合重排; 加 shortcut	$T \rightarrow (T + 1)/2$; H, W 各减半
CNN 上采样	卷积生成所需通道 → 通道重排到指定时空维度	指定轴放大 2 倍; 时间放大后删除最前面 1 帧
NA Decoder 上采样	Linear 生成所需通道 → 通道重排到指定时空维度	指定轴放大 2 倍; 时间放大后删除最前面 1 帧
空间 Unpatchify	将通道重排回每帧的 4×4 像素块	H, W 各乘以 4; C 除以 16; T 不变

下采样中的主分支先通过卷积调整通道，再将局部时空位置并入通道维；shortcut 则对输入做同样的重排，并通过分组平均匹配输出通道。时空采样采用联合重排。

残差块、归一化与分块

- **CNN 残差块**：主分支主要执行两组“PixelNorm → SiLU → 3×3×3 卷积”，再加 shortcut，保持 T, H, W 。
- **PixelNorm**：在每个时空位置上，沿通道维做 RMS 归一化。
- **时间边界**：Encoder 的因果卷积通过复制首帧完成左侧时间填充；含时间下采样的重排模块还会额外复制首帧 1 次。Decoder 则在时间展开后删除前导帧，使整体长度满足 $T_{\text{out}} = 8(T_z - 1) + 1$ 。
- **分块执行**：LTX 支持带重叠区域的 tiling。这里的 $81 = 1 + 10 \times 8$ 表示时间压缩关系；实际解码需要保留跨时间上下文，不能将 11 个 latent 时间位置分别独立解码后直接拼接。

5. FLUX 3 Video VAE

接着来看 FLUX 3 Action 的视频 VAE。它采用因果 **NA Transformer Encoder** + 非因果 **NA Transformer Decoder**，时间压缩倍率为 4，空间高宽各压缩 32 倍，latent 通道数为 96。

同样以 81 帧、480×832 的 RGB 视频为例，形状统一写作 $[C, T, H, W]$ ，省略 $B=1$ ：

整段视频： $[3, 81, 480, 832] \rightarrow \text{latent} : [96, 21, 15, 26] \rightarrow \text{重建视频} : [3, 81, 480, 832]$ 。

$$T_z = 1 + \frac{81 - 1}{4} = 21, \quad H_z = \frac{480}{32} = 15, \quad W_z = \frac{832}{32} = 26.$$

5.1 Block 与 Attention

FLUX 3 的 VAE 也使用了 Neighborhood Attention，统一使用 $5 \times 5 \times 5$ 窗口，还加了 QK Norm 与三维 RoPE；Encoder 使用时间因果 Attention，Decoder 使用非因果 Attention。

特征通道	Attention 头数	每头维度	窗口：时间 × 高 × 宽
256	4	64	$5 \times 5 \times 5$
512	8	64	$5 \times 5 \times 5$
1024	16	64	$5 \times 5 \times 5$
2048	32	64	$5 \times 5 \times 5$

单层 NA 中，Encoder 在时间上只关注当前位置及最多前 4 个位置；Decoder 在非边界处关注前 2 个、当前、后 2 个位置，边界处将窗口向序列内部平移。单帧输入使用 5×5 二维 NA。

5.2 实际分块与上下文

编码分块：按 45 帧一块、每次前进 44 帧执行，相邻块重叠 1 帧，各块独立进入 Encoder。对于经典的 81 帧，会先复制末帧，将 81 帧补到 89 帧：

第 1 块：第 1~45 帧

第 2 块：第 45~89 帧

每块编码得到 12 个 latent 时间位置。保留首块全部输出，后续块丢弃首个 latent 时间位置，再拼接并按原视频长度裁剪：

$$12 + (12 - 1) = 23 \rightarrow \text{保留前 21 个时间位置.}$$

5.3 Encoder：跟踪一个 45 帧视频块

入口按 $1 \times 4 \times 4$ 划分像素块并投影到 256 通道，实现为 kernel 和 stride 均为 $1 \times 4 \times 4$ 的 Conv3d。随后进行三次空间合并，再在低空间分辨率上完成两次时间合并。

顺序	模块与操作	输出形状
输入	当前视频块	$[3, 45, 480, 832]$
入口	$1 \times 4 \times 4$ Conv3d，步长 (1, 4, 4)， $3 \rightarrow 256$	$[256, 45, 120, 208]$
第一级	NA Block $\times 1$ ，256 通道 $\rightarrow$ 空间合并， $256 \rightarrow 512$	$[512, 45, 60, 104]$
第二级	NA Block $\times 4$ ，512 通道 $\rightarrow$ 空间合并， $512 \rightarrow 1024$	$[1024, 45, 30, 52]$
第三级主干	NA Block $\times 8$ ，1024 通道 $\rightarrow$ 空间合并， $1024 \rightarrow 2048$	$[2048, 45, 15, 26]$
第一次时间压缩	附加 NA Block $\times 1 \rightarrow$ 时间合并， $45 \rightarrow 23$	$[2048, 23, 15, 26]$
第四级主干	NA Block $\times 8$ ，2048 通道	$[2048, 23, 15, 26]$

顺序	模块与操作	输出形状
第二次时间压缩	附加 NA Block $\times 1 \rightarrow$ 时间合并, 23 $\rightarrow$ 12	[2048, 12, 15, 26]
出口	逐位置 Linear, 2048 $\rightarrow$ 192	[192, 12, 15, 26]

空间压缩由入口的 4 倍和三次 2 倍空间合并组成:

$$4 \times 2 \times 2 \times 2 = 32.$$

时间长度则依次为:

$$45 \rightarrow 23 \rightarrow 12.$$

出口沿通道拆成两组, 各为 [96, 12, 15, 26], 只取前一组作为 μ 。随后对 μ 使用模型保存的逐通道均值和标准差归一化。

5.4 Decoder: 逐级恢复视频

先撤销 latent 的逐通道归一化, 再通过 Linear 投影、NA Block 和时空展开, 恢复 RGB 视频。形状统一为 [C, T, H, W], 省略 batch 维。

顺序	模块与操作	输出形状
输入	撤销归一化后的 latent	[96, 21, 15, 26]
入口	逐位置 Linear, 96 $\rightarrow$ 2048	[2048, 21, 15, 26]
第一级时间展开	NA Block $\times 8 \rightarrow$ 时间展开, 21 $\rightarrow$ 41 $\rightarrow$ NA Block $\times 1$	[2048, 41, 15, 26]
第一级空间展开	空间展开, 2048 $\rightarrow$ 1024	[1024, 41, 30, 52]
第二级时间展开	NA Block $\times 8 \rightarrow$ 时间展开, 41 $\rightarrow$ 81 $\rightarrow$ NA Block $\times 1$	[1024, 81, 30, 52]
第二级空间展开	空间展开, 1024 $\rightarrow$ 512	[512, 81, 60, 104]
第三级	NA Block $\times 4 \rightarrow$ 空间展开, 512 $\rightarrow$ 256	[256, 81, 120, 208]
第四级	NA Block $\times 1$, 保持 256 通道	[256, 81, 120, 208]
出口	逐位置 Linear, 256 $\rightarrow$ 48	[48, 81, 120, 208]
Unpatchify	将 48 $=3 \times 4 \times 4$ 个通道重排为 RGB 像素块	[3, 81, 480, 832]

每次时间展开满足 $T \rightarrow 2T - 1$, 因此时间长度为:

$$21 \rightarrow 41 \rightarrow 81.$$

5.5 上下采样与首帧对齐

操作	具体实现	形状变化
入口 Patch Embedding	$1 \times 4 \times 4$ Conv3d, 步长 $1 \times 4 \times 4$, $3 \rightarrow 256$	H, W 各除以 4; T 不变
空间合并	拼接 2×2 相邻位置: $C \rightarrow 4C \rightarrow \text{LayerNorm} \rightarrow \text{Linear } 4C \rightarrow 2C$	H, W 各减半; C 翻倍
时间合并	奇数长度时在开头复制首位置 $\rightarrow$ 两位置拼接 $\rightarrow \text{LayerNorm} \rightarrow \text{Linear } 2C \rightarrow C \rightarrow$ 加两位置的均值 shortcut	$T \rightarrow \lceil T/2 \rceil$; C, H, W 不变
空间展开	$\text{LayerNorm} \rightarrow \text{Linear } C \rightarrow 4C_{\text{out}} \rightarrow$ 重排成 2×2 空间位置, 其中 $C_{\text{out}} = C/2$	H, W 各翻倍; C 减半
时间展开	$\text{LayerNorm} \rightarrow \text{Linear } C \rightarrow 2C \rightarrow$ 加输入在通道维复制两份的 shortcut $\rightarrow$ 重排到时间维 $\rightarrow$ 删除首位置	$T \rightarrow 2T - 1$; C, H, W 不变
空间 Unpatchify	将每个位置的 48 个通道重排为 $3 \times 4 \times 4$ 像素块	H, W 各乘以 4; 输出 3 通道

执行顺序:

- Encoder 第三级: 空间合并 $\rightarrow$ 附加 NA Block $\rightarrow$ 时间合并; 第四级在主干后执行附加 NA Block $\rightarrow$ 时间合并。
- Decoder 前两级: 时间展开 $\rightarrow$ 附加 NA Block $\rightarrow$ 空间展开。

首帧对齐: 编码时通过复制首位置补齐奇数长度, 解码时通过删除展开后的首位置恢复对齐。单帧也经过这些模块, 但长度仍为 1:

Encoder: $1 \rightarrow$ 复制成 2 $\rightarrow$ 合并成 1

Decoder: $1 \rightarrow$ 展开成 2 $\rightarrow$ 删除首位置后剩 1

两次时间展开后, 输出长度为:

$$T_{\text{out}} = 2(2T_z - 1) - 1 = 4(T_z - 1) + 1.$$